

Chapters 4 and 5 — Draft for TUM ICI Project Report

Title context: A Web-Based Modular Enterprise Resource Planning (ERP) System for SMEs: A Case Study of Macike Enterprise

Author: Mike Mutuma Gikunda (BSIT/611J/2022)

Note: Apply ICI formatting in Word: Times New Roman 12 pt (body), 14 pt chapter titles, 1.5 line spacing, margins 1" (1.25" left), page numbers bottom centre. Replace all **[Insert Figure X.X]** placeholders with your screenshots. References should be completed in **APA 7th** in your final document.

CHAPTER FOUR: RESEARCH FINDINGS AND DISCUSSION

4.1 Introduction

This chapter presents the research findings arising from the design, implementation, and evaluation of the web-based modular Enterprise Resource Planning (ERP) system developed for Small and Medium Enterprises (SMEs), using **Macike Enterprise** as the case study. The chapter is structured to align each finding with the specific objectives set out in Chapter One. The software artifact was engineered using the **Django** web framework following the **Model-View-Template (MVT)** pattern, with modular applications for user administration, dashboard and settings, inventory management, and sales—including **M-Pesa (Safaricom Daraja)** integration for sandbox STK Push and asynchronous payment confirmation via callback.

The findings combine (i) outcomes of requirements alignment with the implemented modules, (ii) evidence from the running system (user interface flows, data integrity behaviour, and reporting), and (iii) structured system testing and user acceptance-oriented checks against the functional and non-functional requirements defined in Chapter Three. Where quantitative claims depend on deployment context (e.g., concurrent load), the discussion states the environment explicitly—**local development**, **SQLite** for rapid iteration, and **production-style deployment** using **Gunicorn** and **Nginx** on a Linux Virtual Private Server (VPS) with public access at **macike.space**.

4.2 Research Setting and Evaluation Approach

The case organization, **Macike Enterprise**, provided the operational context for requirements validation: retail-oriented workflows involving stock intake, shelf sales, and receipting, with a need for **role separation** between administration, inventory management, cashiering, and audit-style review. Evaluation of the artifact followed a **design-science / applied research** orientation consistent with Chapter Three: the system was assessed against the stated objectives and research questions through demonstration of working features, inspection of database behaviour (transactional stock deduction, referential links between sales and inventory entities), and review of security and usability characteristics appropriate to a web-based financial application.

Primary evidence for Chapter Four is drawn from the **implemented codebase** and **running application**, supplemented by test-case execution records (to be attached or referenced in appendices as **test evidence screenshots**). This approach is appropriate for an ICT artefact-driven project where the contribution is a **working system** whose correctness is

demonstrated through observable behaviour and structured testing rather than solely through statistical survey analysis.

4.3 Presentation of Findings

4.3.1 Objective One: Analysis of SME Operational Workflows and Data Redundancy Risks

Objective (from Chapter One): To critically analyse the current operational workflows of SMEs in Mombasa, specifically identifying points of data redundancy and bottlenecks caused by manual synchronization of records.

Findings.

Field-oriented requirements for the case study confirmed the problem pattern described in the proposal: when sales records and inventory records are maintained in **disjoint ledgers or spreadsheets**, stock figures and cash positions diverge over time. The implemented ERP addresses this structurally by enforcing a **single relational model** in which a **sales invoice** is tied to **line items**, **products**, and (where applicable) **customers**, and in which stock movements are recorded through **FIFO batch consumption** and **audit logs** rather than through manual post hoc adjustment.

The system's POS and inventory modules make the "synchronization bottleneck" visible in process terms: a cashier cannot complete a **cash or card** sale without passing stock validation, and a completed **M-Pesa** sale transitions to a **paid** state only after **Daraja callback confirmation**, reducing the risk of marking revenue without a confirmed payment event. This does not remove all operational risk (human error at goods receipt, physical shrinkage), but it removes the **software-level** inconsistency where sales and stock updates are independent edits.

Discussion.

The finding supports the proposition that **workflow integration**—not merely digitization—is required. The ERP does not only store products and sales; it connects them through **foreign keys** and **transactional updates**, which is the technical counterpart of "one source of truth" emphasized in the literature review.

Evidence placeholder.

[Insert Figure 4.1: Dashboard home showing recent sales and low-stock indicators tied to integrated data.]

[Insert Figure 4.2: Inventory product list illustrating consolidated stock view versus fragmented spreadsheet records.]

4.3.2 Objective Two: Design of a Normalized Relational Schema Enforcing Referential Integrity

Objective: To design a normalized relational database schema (utilizing PostgreSQL in the proposal specification) that enforces referential integrity among Sales, Inventory, and Customer entities.

****Findings.****

The implemented schema follows **third normal form (3NF)** principles as outlined in Chapter Three: categorical and supplier data are separated (**ProductCategory**, **Supplier**), the inventory master is modeled in **Product**, transactional headers and lines are separated (**SalesInvoice**, **SalesLineItem**), and stock movements are traceable through **StockBatch** and **StockAuditLog**. Customer linkage is supported through the **Customer** entity and optional association on invoices, enabling repeat-buyer tracking without duplicating customer attributes on every sale header beyond display fields required at checkout.

Role and authentication data reside in **CustomUser**, extending Django's user model with enumerated **roles** (Administrator, Manager, Cashier, Auditor) to support **RBAC** enforcement at the view layer. M-Pesa integration persists gateway identifiers and callback payloads in **MpesaTransaction**, linked to **SalesInvoice**, supporting reconciliation and audit.

The proposal specifies **PostgreSQL** for production-grade ACID guarantees; the repository supports this stack in documentation, while local development may use **SQLite** for convenience. The **logical** integrity design (keys, relationships, transactional updates) is consistent across engines under Django's ORM, with PostgreSQL recommended for concurrent production workloads.

****Discussion.****

Normalization materially reduces update anomalies: product description and pricing changes are anchored in **Product** rather than duplicated across historical rows beyond the **unit_price** snapshot captured on each **SalesLineItem**, which is appropriate for retail audit trails. The schema therefore balances **integrity** with **historical accuracy** at the line-item level.

****Evidence placeholder.****

[Insert Figure 4.3: Entity-relationship diagram exported from documentation or modeling tool, aligned to implemented models.]

[Insert Figure 4.4: Django Admin or database view showing foreign-key relationships for a sample invoice and line items.]

4.3.3 Objective Three: Implementation of a Modular MVT Architecture (RBAC, Inventory, Transactional Invoicing, POS)

****Objective:**** To implement a modular architecture based on MVT, delivering secure authentication (RBAC), real-time inventory tracking, and transactional invoicing including POS operations.

****Findings.****

The system is organized into Django **apps**—**accounts**, **dashboard**, **inventory**, and **sales**—consistent with separation of concerns. **RBAC** is enforced through decorators restricting views by role (e.g., cashier versus auditor access patterns). The **inventory** module supports category and supplier maintenance, product cataloguing, restocking that creates **FIFO batches**, and listing with **low-stock** signalling relative to **reorder_level**. The **sales** module implements **POS checkout** with VAT and discount handling driven by **AppSettings**, generates **HTML receipts** and **PDF invoices**, and provides **sales**

reporting** with payment-method breakdowns and recent M-Pesa entries.

M-Pesa STK Push is initiated server-side through **Safaricom Daraja** sandbox endpoints; payment completion is **asynchronous**, handled by a **callback URL** mapped to **/sales/mpesa/callback/**. Production deployment guidance includes **Nginx** as a reverse proxy to a **Gunicorn** UNIX socket and static file serving for collected assets, supporting HTTPS termination and stable public endpoints required by the gateway.

Discussion.

The modular structure achieved the intent stated in the proposal: avoid monolithic “platform lock-in” while still delivering ERP-like cohesion through a shared database and shared settings. The POS path demonstrates **transactional discipline**: stock finalization for M-Pesa occurs after payment confirmation, aligning financial state with inventory state.

Evidence placeholder (required types per ICI guidelines).

[Insert Figure 4.5: Login screen—authentication.]

[Insert Figure 4.6: User management / role concept (admin list or role-restricted screen).]

[Insert Figure 4.7: Product create / inventory data entry form.]

[Insert Figure 4.8: Restock / batch intake form.]

[Insert Figure 4.9: POS checkout screen.]

[Insert Figure 4.10: Receipt / invoice view after successful sale.]

[Insert Figure 4.11: Sales report / analytics output.]

[Insert Figure 4.12: Validation or error state—e.g., insufficient stock message.]

[Insert Figure 4.13: M-Pesa callback success reflected in receipt or transaction record (mask sensitive credentials in screenshots).]

4.3.4 Objective Four: System Validation (Testing, Security Awareness, Usability)

Objective: To perform rigorous validation through unit testing and user acceptance testing (UAT), evaluating performance, security against common web vulnerabilities, and usability relative to legacy manual processes.

Findings.

Structured **test cases** defined in Chapter Three (e.g., authentication, SKU uniqueness, stock deduction, insufficient stock handling, role access, reporting) provide a repeatable validation matrix. Execution outcomes indicate:

- **Functional correctness:** Checkout flows create consistent invoice totals; stock deductions align with paid sales for cash/card and with confirmed M-Pesa callbacks for mobile payments.
- **Data integrity:** FIFO consumption raises explicit errors when batch quantities are insufficient, preventing silent negative inventory in the batch layer; product quantity is adjusted consistently with audit logging for sales and restocks.
- **Security posture (baseline):** Django’s built-in protections (CSRF on browser forms, ORM parameterization reducing **SQL injection** risk, password hashing) provide a mainstream baseline; deployment with HTTPS via **Nginx/Certbot** improves transport security for production.
- **Usability:** Bootstrap-based responsive layouts reduce training friction; role-based

redirection and messaging communicate permission boundaries.

****Limitations of validation (transparent reporting).****

Longitudinal operational metrics (multi-year shrinkage reduction) and high-concurrency stress testing were not within the academic project's resource envelope; findings should be interpreted as ****engineering validation**** of the artefact rather than a full enterprise load test.

****Discussion.****

For SME contexts, "rigorous" validation appropriately emphasizes ****traceability**** (who changed stock, when, and why) and ****correct transactional transitions**** (pending payment → paid/failed). This matches the project's ACID-oriented rationale better than abstract benchmarking alone.

****Evidence placeholder.****

[Insert Table 4.1: Sample test case matrix with Pass/Fail results and dates.]

[Insert Figure 4.14: Excerpt of stock audit log demonstrating accountability.]

4.4 Chapter Summary

This chapter presented findings mapped to the four specific objectives: (i) integrated workflows reduce manual synchronization failure modes common in disjoint record-keeping; (ii) the implemented relational schema enforces referential integrity and supports auditability; (iii) modular MVT implementation delivers RBAC, FIFO-aware inventory, POS, invoicing, reporting, and M-Pesa integration with deployment guidance suitable for a VPS; and (iv) structured testing supports claims of functional reliability and baseline security, within stated scope limits. Chapter Five synthesizes these findings into conclusions, explicitly addresses the research questions, and states recommendations and avenues for further work.

CHAPTER FIVE: SUMMARY, CONCLUSIONS AND RECOMMENDATIONS

5.1 Introduction

This final chapter consolidates the outcomes of the research project by summarizing the findings presented in Chapter Four, drawing conclusions that directly respond to the research questions formulated in Chapter One, and presenting actionable recommendations for Macike Enterprise and similar SMEs. It concludes with suggestions for further study that extend the current artefact without contradicting the defined project scope.

5.2 Summary of Findings

5.2.1 General observation

The project produced a ****working, modular ERP web application**** tailored to retail-oriented SME operations, demonstrating that a ****framework-first**** implementation on Django can deliver integrated inventory and sales capabilities without adopting a heavyweight monolithic ERP platform. The case study configuration (Macike Enterprise) grounds the design in realistic

requirements: multi-user operation, VAT-aware pricing, receipting, and optional mobile money checkout.

5.2.2 Objective one: Workflow analysis and redundancy risks

Manual and spreadsheet-based operations were found to be structurally prone to **desynchronization** between sales and stock records. The implemented system mitigates this by binding sales, stock movements, and audit trails in a single database transaction model appropriate to each payment channel.

5.2.3 Objective two: Normalized schema and integrity

The database design separates master data, transactional headers and lines, batch inventory, and payment gateway records, supporting **3NF** goals and practical audit requirements. The architecture aligns with the proposal's PostgreSQL orientation for production deployments.

5.2.4 Objective three: Modular implementation and POS/M-Pesa

RBAC, inventory, POS, PDF receipts, dashboard analytics, and **Daraja STK Push** with callback processing were implemented in a decoupled app structure. Deployment documentation (**Gunicorn + Nginx**) supports hosting on a VPS with a public domain (**macike.space**), which is essential for HTTPS callbacks in mobile money integrations.

5.2.5 Objective four: Validation

Testing and structured walkthroughs support claims of functional reliability and baseline web security; limitations include limited longitudinal business metrics and limited large-scale concurrency testing.

5.3 Conclusions (Research Questions)

The conclusions below address the research questions in Chapter One in integrated paragraphs rather than yes/no fragments.

RQ1 (manual systems, redundancy, latency). Disjoint manual and heterogeneous digital records contribute materially to redundancy and operational latency because the same facts must be re-entered or reconciled across multiple stores of truth. The project demonstrates that integrating sales and inventory under relational constraints reduces these failure modes by making stock updates a consequence of validated checkout processes rather than a separate administrative chore.

RQ2 (Django modular architecture, ACID, bloat mitigation). A modular Django architecture can implement enterprise-relevant transactional rules—particularly **atomic checkout and stock finalization**—without importing unused modules from large ERP platforms. The “bloat mitigation” outcome is evidenced by a lean module set focused on authentication, inventory, sales, and reporting, while still supporting payment integration.

RQ3 (RBAC and Django security middleware). Role-based restrictions and Django's security stack provide a credible baseline for protecting financial workflows from common web threats

and unauthorized role escalation, provided deployment maintains HTTPS, patching, and sound secret management. Security is never “complete,” but the design aligns with mainstream practice for small deployed systems.

RQ4 (framework-based vs platform-based). Relative to platform-based ERPs, the framework-based solution trades breadth of prebuilt modules for **customizability and transparency**. Resource utilization is inherently dependent on traffic and hosting; the important comparative finding for SMEs is **maintainability and fit-to-scope** rather than raw benchmark supremacy.

5.4 Recommendations

1. **Production database:** Migrate production from SQLite (if still used on the server) to **PostgreSQL** to align with the proposal and improve concurrency robustness.
2. **Secrets and keys:** Store **Django SECRET_KEY**, **Daraja credentials**, and database passwords in environment variables or a secrets manager; never commit them to version control.
3. **HTTPS and headers:** Maintain **TLS** certificates (e.g., Certbot) and review security headers and cookie settings for production `DEBUG=False`.
4. **Backups:** Schedule automated backups for the database and media (if introduced); supplement CSV exports with full DB dumps for disaster recovery.
5. **Operational training:** Provide short role-based training (cashier vs manager) emphasizing **M-Pesa pending vs paid** states and stock correction procedures through audited restocks.
6. **Monitoring:** Add application logging and basic uptime monitoring on the VPS to detect **502** proxy failures early (common when Gunicorn socket permissions or service restarts misalign).

5.5 Suggestions for Further Study

1. **Offline-first POS:** Explore Progressive Web App (PWA) techniques or local edge sync for intermittent connectivity contexts emphasized in Kenyan SME reality.
2. **Double-entry accounting module:** Extend financial controls with generalized ledger postings while preserving inventory traceability.
3. **Procurement and purchase orders:** Add supplier purchase workflows and goods-received processing integrated with batch costing.
4. **Advanced analytics:** Incorporate demand forecasting using historical sales series (seasonality, promotions).
5. **Fraud and anomaly detection:** Apply rule-based alerts (unusual void patterns, repeated failed M-Pesa attempts) for internal control.
6. **Performance engineering:** Conduct formal load testing on PostgreSQL with realistic concurrent cashier sessions and document scalability limits.

5.6 Chapter Summary

The project successfully delivered a modular Django ERP aligned with Macike Enterprise’s core needs: integrated inventory and sales, RBAC, FIFO-aware stock handling, reporting, and M-Pesa sandbox integration with asynchronous confirmation. Conclusions support the central thesis that a **framework-based, scope-controlled ERP** can address SME data integrity gaps without the cost and rigidity of large proprietary or monolithic open-source suites. Recommendations focus on production hardening—especially database choice, secrets management, HTTPS operations, and backups—while further study outlines responsible extensions beyond the current academic scope.

REFERENCES (add / verify in APA 7th)

Students must ensure every citation in Chapters 4–5 appears here. Examples to verify and format consistently:

- Django Software Foundation. (2024). *Django documentation*. <https://docs.djangoproject.com/>
- OWASP. (2021). *OWASP Top Ten*. <https://owasp.org/www-project-top-ten/>
- PostgreSQL Global Development Group. (2024). *PostgreSQL documentation*. <https://www.postgresql.org/docs/>
- (Add Safaricom Daraja API documentation citation if referenced in text.)

APPENDIX REMINDERS (ICI)

For the **final report**, include updated appendices as required: questionnaire summary, budget, work plan, **program code excerpts** (repository link or selected modules), and **screenshots** matching Figure placeholders above.